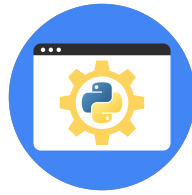


# Course One

## Foundations of Data Science



### Instructions

Use this PACE strategy document to record decisions and reflections as you work through this end-of-course project. You can use this document as a guide to consider your responses and reflections at different stages of the data analytical process. Additionally, the PACE strategy documents can be used as a resource when working on future projects.

### Course Project Recap

Regardless of which track you have chosen to complete, your goals for this project are:

- Complete the questions in the Course 1 PACE strategy document
- Answer the questions in the Jupyter notebook project file
- Complete coding prep work on project's Jupyter notebook
- Summarize the column Dtypes
- Communicate important findings in the form of an executive summary

### Relevant Interview Questions

Completing the end-of-course project will help you respond these types of questions that are often asked during the interview process:

**Describe the steps you would take to clean and transform an unstructured data set.**

**A strong structure for this answer has roughly 6 steps:**

#### 1. Understand the data first (before touching anything)

Review any data dictionary, understand what each column represents, and clarify the business question the data needs to answer. You can't clean data meaningfully without knowing what "correct" looks like for that field.

#### 2. Load and inspect the structure

Load it into a DataFrame and run structural checks — `.info()`, `.shape`, `.dtypes`, `.head()`. This is exactly what you did with the taxi data: confirming column count, data types, and getting a first look at what you're working with.

#### 3. Check for missing data

Use `.isnull().sum()` to find nulls. Then decide, column by column, whether to:

- Drop rows/columns (if missing data is minimal or the column is unusable)
- Impute values (mean/median/mode, or a more sophisticated method)
- Leave a flag/placeholder if "missing" itself is meaningful

#### 4. Check for invalid, inconsistent, or outlier values

This is exactly what you did with `.describe()` and sorting — catching negative fares, 0-passenger trips, invalid codes like `RatecodeID = 99`, and extreme outliers like the \$1,200 `total_amount`. In an interview, mention specific detection techniques: summary statistics, sorting max/min, box plots, or z-scores/IQR for outlier detection.

#### 5. Fix data types and formatting

Convert text-stored dates to actual `datetime` objects, fix inconsistent capitalization/spacing in strings, standardize categorical labels (e.g., "NY" vs "New York" vs "ny"), and remove duplicate rows.

#### 6. Transform for analysis

This is feature engineering — deriving new, more useful variables. In your case: calculating `trip_duration` from pickup/dropoff timestamps, or extracting `pickup_hour/day_of_week`. Also includes things like encoding categorical variables for modeling, or aggregating/reshaping data if needed.

A good closing line for the interview: mention that you'd document what you changed and why (a data cleaning log), since stakeholders and teammates need to trust and reproduce your process — this shows maturity beyond just "I ran some pandas functions."

### **What specific things might you look for as part of your cleaning process?**

#### 1. Missing values

- Nulls/NaNs in any column (`.isnull().sum()`)
- Blank strings or placeholder values that *look* filled but aren't real data (e.g., "N/A", "unknown", empty strings, or sentinel values like `-999`)

#### 2. Duplicate records

- Fully duplicated rows (`.duplicated().sum()`)
- Near-duplicates that might represent the same real-world entity with slight formatting differences (e.g., "NYC" vs "New York City")

#### 3. Inconsistent formatting

- Inconsistent capitalization or spacing in text/categorical fields
- Inconsistent date formats (`MM/DD/YYYY` vs `YYYY-MM-DD`)
- Mixed units (e.g., some distances in miles, others in km, if merging datasets)

#### 4. Incorrect data types

- Numbers stored as text (`object` dtype instead of `int/float`)
- Dates stored as text instead of `datetime` — exactly what you ran into with `tpep_pickup_datetime`

#### 5. Invalid or implausible values

- Values outside a logical range — like the **negative fares** and **negative taxes** you found

- Categorical codes that don't match the data dictionary — like `RatecodeID = 99`
- Logical inconsistencies between columns — e.g., `0 trip_distance` but a real elapsed time between pickup/dropoff, which you specifically caught

## 6. Outliers

- Extreme values far outside the normal range, like the **\$1,200 total\_amount** — worth flagging even if not strictly "wrong," since they can distort averages, std deviation, and model training

## 7. Structural issues

- Irrelevant or redundant columns (like that leftover `Unnamed: 0` index column)
- Columns that are technically numeric but are actually categorical/ID codes and shouldn't be treated as continuous (`VendorID`, `PULocationID`, etc. — you nailed this concept earlier)

## 8. Class imbalance (if relevant to modelling later)

- For categorical target variables, checking if one class vastly outnumbers another, which can bias a model

Good interview delivery tip: rather than reciting this as a dry list, anchor 2–3 of these to a real example from a project you've done (like the taxi dataset) — it shows you've actually *practiced* this, not just memorized a checklist. E.g., *"In a recent project, I found trips with \$0 fare\_amount but real trip distances, which told me the fare data — not the trip itself — was likely corrupted."*

**What are some of the outliers, anomalies, or unusual things you might look for in the data cleaning process that might impact analyses or ability to create insights?**

### 1. Statistical outliers

- Values far outside the typical range (e.g., using IQR — values below  $Q1 - 1.5 \times IQR$  or above  $Q3 + 1.5 \times IQR$  — or z-scores beyond  $\pm 3$  standard deviations)
- In your taxi data: the **\$1,200 total\_amount**, way beyond the \$6.50–\$14.50 typical range you found in `fare_amount`

### 2. Impossible/illogical values

- Values that break real-world logic, regardless of statistical distribution — like **negative fares, negative taxes, or negative total\_amount**. A fare can be low, but it can't be negative in a normal transaction.
- **0-passenger trips** — a taxi can't carry zero people

### 3. Internal inconsistencies between related columns

- This is a more advanced one to mention, and it shows deeper thinking: values that are individually plausible but **contradict each other**. You caught this exact pattern — trips with **0 trip\_distance but real elapsed time** between pickup and dropoff (e.g., 13 minutes). Neither value alone looks "wrong," but together they reveal a data recording error.

### 4. Invalid categorical/code values

- Category codes falling outside the expected set defined by the data dictionary — like **RatecodeID = 99**, when valid codes only go up to ~6

### 5. Suspicious repetition or default values

- The same "placeholder" value showing up unusually often (e.g., a string of trips all showing exactly \$0.00, or a max value like 999.99 appearing suspiciously often) can indicate a system default or error code rather than real data, rather than a true measurement

#### 6. Skew between mean and median

- A large gap between mean and median (like your fare\_amount example: mean \$13.03 vs. median \$9.50) signals the distribution is being pulled by outliers — worth flagging even before looking at individual rows

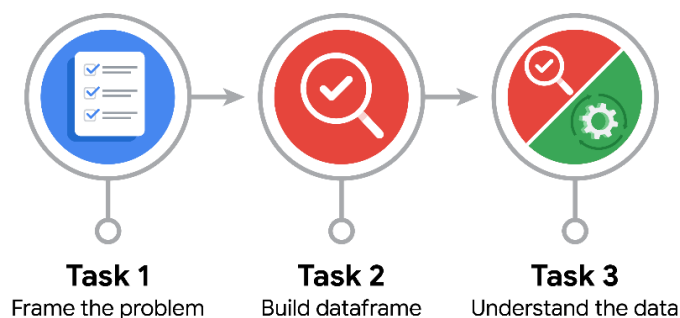
#### **Why this matters for insights/analysis (the "so what" the question is really asking for):**

- Outliers can **distort summary statistics** — inflate the mean, inflate standard deviation, making "typical" values look misleading
- They can **skew a regression/predictive model** — a model trained on data including a \$1,200 fare outlier might overweight extreme cases and generalize poorly to typical trips
- They can **violate assumptions** of certain statistical tests (e.g., normality assumptions in some hypothesis tests)
- Left unaddressed, they can lead to **wrong business conclusions** — e.g., telling a stakeholder "average fare is \$16" when the real typical experience is closer to \$9.50, because a couple of extreme trips dragged the average up

**Strong closing point for an interview:** mention that outliers aren't automatically "wrong" and shouldn't be blindly deleted — the right move is to *investigate* them first (are they real rare events, or data errors?), then decide whether to correct, remove, cap, or keep them with a flag, depending on what you find and what the analysis needs.

## Reference Guide

This project has three tasks; the visual below identifies how the stages of PACE are incorporated across those tasks.



## Data Project Questions & Considerations



### PACE: Plan Stage

- How can you best prepare to understand and organize the provided information?

Start by reviewing any available data dictionary to understand what each column represents, its expected data type, and units of measurement. Then clarify the business question driving the project (e.g., predicting taxi fare amount) so it's clear which variables are likely relevant. From there, load the data into a structured format (like a pandas DataFrame) and run a structural review — checking shape, column dtypes, and missing values — to get an accurate picture of what you're working with before any deeper analysis begins.

- What follow-along and self-review codebooks will help you perform this work?

A codebook (or data dictionary) that defines each variable, its expected values/ranges, and units is essential — it acts as the reference point for judging whether a value is valid or an error. Beyond that, keeping a personal "cleaning log" — documenting decisions like which rows were dropped, which values were flagged, and why — serves as a self-review tool, letting you retrace your reasoning and explain choices to stakeholders later. Style/coding standard guides (e.g., PEP 8 for Python) are also useful for keeping notebooks clean and readable for teammates reviewing your work.

- What are some additional activities a resourceful learner would perform before starting to code?



A resourceful learner would research the context behind the dataset — for example, understanding how NYC taxi fares are actually calculated (base fare, per-mile/per-minute rates, surcharges) so they can judge whether values look realistic. They'd also look at what similar projects or public analyses have found with this type of data, to anticipate common data quality issues. Sketching out a rough plan or pseudocode for the cleaning/analysis steps before touching any code — rather than diving in reactively — helps avoid wasted work. Finally, setting up the environment properly (confirming libraries are installed, organizing file paths) prevents avoidable friction once coding starts.

**PACE: Analyze Stage**

- Will the available information be sufficient to achieve the goal based on your intuition and the analysis of the variables?

Yes, overall the dataset appears sufficient to work toward the goal of predicting fare amount. Key variables like `trip_distance`, pickup/dropoff timestamps (once converted to datetime), and `passenger_count` have a logical, direct relationship to how taxi fares are calculated, making them strong candidate predictors. That said, the data does have quality issues — negative fares, 0-distance trips, and outliers — that need to be addressed first, or they could weaken the model's reliability. Additional context, like weather or traffic conditions, could further improve predictions, but isn't available in this dataset.

- How would you build summary dataframe statistics and assess the min and max range of the data?

I'd use `df.describe()` to generate summary statistics (count, mean, std, min, 25/50/75th percentiles, and max) across all numeric columns in a single view. To dig deeper into the extremes specifically, I'd sort individual columns using `df.sort_values(by='column_name', ascending=False)` and `ascending=True` to see the top and bottom values directly, rather than relying only on the single min/max number — this helps reveal whether extreme values are isolated outliers or part of a broader pattern (e.g., several trips sharing the same implausible value).

- Do the averages of any of the data variables look unusual? Can you describe the interval data?

Yes — for `fare_amount`, the mean (\$13.03) was noticeably higher than the median (\$9.50), signaling the average is being pulled upward by a small number of extreme outliers (like the \$999.99 max fare) rather than reflecting a "typical" trip. This is a common sign of a right-skewed distribution. For interval data (continuous numeric variables like `fare_amount`, `trip_distance`, and `tip_amount`), the gap between the 25th and 75th percentiles shows where the bulk of "normal" values fall — for fares, that middle 50% of trips ranged from about \$6.50 to \$14.50 — which is a more reliable picture of typical fares than the mean alone, given the skew.

**PACE: Construct Stage**

**Note:** The Construct stage does not apply to this workflow. The PACE framework can be adapted to fit the specific requirements of any project.



### PACE: Execute Stage

- Given your current knowledge of the data, what would you initially recommend to your manager to investigate further prior to performing exploratory data analysis?

I'd recommend investigating the records with negative values in `fare_amount`, `extra`, `mta_tax`, and `total_amount` — since these aren't just statistical outliers, they're logically impossible for a standard fare transaction, suggesting either refunds, corrections, or a data pipeline issue that's worth understanding before deciding how to handle them. I'd also recommend confirming with the data source or team what `RatecodeID = 99` actually represents, since it falls outside the documented rate codes and could either be a legitimate "other" category or an error code. Getting clarity on both would prevent us from either wrongly deleting valid data or leaving invalid data in the analysis.

- What data initially presents as containing anomalies?

Several fields stood out: `fare_amount`, `extra`, `mta_tax`, and `total_amount` all contain negative values, which aren't logically valid for standard trips. `trip_distance` and `passenger_count` both include 0 values — a 0-mile trip or a trip with 0 passengers doesn't reflect a real taxi ride. `RatecodeID` includes a value of 99, well outside its expected range of documented codes. Finally, `fare_amount`, `tip_amount`, and `total_amount` each contain extreme outliers (e.g., a \$999.99 fare and a \$1,200.29 total) far beyond the typical range seen in the rest of the data.

- What additional types of data could strengthen this dataset?

A few additions would likely improve the ability to predict `fare_amount` accurately:

- Traffic or congestion data** at the time of the trip, since fare and duration are affected by how much traffic a trip encounters
- Weather data**, since rain/snow can affect both demand and trip duration
- A proper zone name/borough lookup** for `PULocationID`/`DOLocationID` (rather than just numeric IDs), making it easier to analyze fare patterns by actual geography (e.g., airport zones vs. Manhattan core)



- **Driver or vehicle-level data** (if available and relevant), to check whether fare or tip patterns vary by those factors
- **A more complete or updated data dictionary/documentation**, to resolve ambiguous codes like `RatecodeID = 99` with certainty rather than guessing

|  |
|--|
|  |
|--|